

**Szegedi Tudományegyetem
Informatikai Tanszékcsoport**

SZAKDOLGOZAT

Dinnyés Balázs Emil

2010

**Szegedi Tudományegyetem
Informatikai Tanszékcsoport**

**Szállítmányozó és raktározó céget irányító és
támogató program**

Szakdolgozat

Készítette:
Dinnyés Balázs Emil
informatika szakos
hallgató

Témavezető:
Dr. Schrettner Lajos
Adjunktus

Szeged
2010

Feladatkiírás

A megadott feladat megfogalmazása: Egy szállítványozó és raktározó cég legfontosabb munkafolyamatait megkönnyítő program.

Egy ilyen cégnek, vállalatnak több lerakata, telephelye van, ez a project egy ilyen, nagyobb lerakatnak a segítségére válik, hiszen csak így van értelme a komputeres támogatásnak - monumentálisabb méretek miatt is.

A segítség,- úgymint raktározással járó, optimális helykihasználást segítő modul; az utazó ügynök (TSP) problémáját, egy exponenciális futási idejű, megoldó modul ($n < 2 \cdot 10^7$), mely a megpakolt tehergépkocsik, és kamionok minimális útmegtételét számítja ki a raktártól és vissza. Ezen kívül a program tartalmazza a helyi alkalmazottak munkáját segítő modult. Ezen programrész a ládapakolási algoritmust kihasználva segíti azok munkáját.

Minden telephelyen van alkalmazott, nem túl magasan iskolázott munkaerő, mely gárda a gépkocsikra való pakolást, az áruk elhelyezését, és bepakolását végzik.

A megoldási mód: A megoldási mód a Java környezet. Grafikus felületen a java.AWT csomag felhasználásával.

Alkalmazott eszközök, módszerek: személyi számítógép. A program úgy lesz megszerkesztve, hogy képes legyen vagy file - ből venni például két várost összekötő legrövidebb út adatát, vagy például a tárolásra alkalmas konténerek tömegét, - vagy közvetlenül is megadhatjuk a programnak.

A konténerekkel (konténerek = m) végzett művelet az egymásra pakolásuk (n) helyre ($n < m$) a lehető legkisebb munkával. A ládapakolás: Tegyük fel, hogy van egy (v) tárgyból álló halmaz, ahol az i . tárgy $s(i)$ méretére $0 < s(i) < 1$. Az összes tárgyat szeretnénk bepakolni minimális számú egységnyi méretű ládába. Minden ládába a tárgyak bármely részhalmaza belefér, amelynek összes mérete nem haladja meg az 1-et. A feladatot az algoritmusok közt jól ismert részletösszeg feladattól vezetem le.

A TSP modul részében a $2^n \cdot n^2$ futási idejű és n^2 tárigényű megoldási módszere, az alapfeladatnak kerül megvalósításra.

Grafikus felületről: A program elindításakor egy kisebb ablakokra osztott teljes képernyő fogadja a felhasználót. Lehet választani az említett három fő funkciót

tartalmazó ablakok, az adatbekérő ablakocská, és az eredmény ablak között. A programot, ha kell, felíratos gombokkal és funkciójaival, illetve menüvel is ellátom.

Tartalmi összefoglaló

A programom, szállítmányozó és raktározó céget irányító és támogató program.

A program eleget tesz a feladat kiírásnak, a következő két részlet kivételével. A TSP modul – ban megadható városok, maximális száma nem 17, hanem 100, a számítási idő egy bizonyos városszám felett kivárhatatlanul sok. A ládapakolás modult nem a részletösszeg feladatból vezettem le, hanem helyette heurisztikusan oldottam meg, a First - fit heurisztikával, de hogy mégis tartsam a kiíráshoz magam, megadok a problémára egy lehetséges NP - teljes megoldást is a fejezetben. Egyébként minden másnak eleget tesz a programom az általam, előzőleg megadott feladatkiírásnak, kezdve a dinamikus programozású moduloktól, a Java AWT csomagjának használatán keresztül, a file műveletekig.

Tartalomjegyék

Feladatkiírás	3
Tartalmi összefoglaló	4
Tartalomjegyék	4
1 Általános megoldási módok	6
1.1 TSP	6
1.1.1 Kapcsolódó problémák	6
1.1.2 Számítási nehézség	7
1.1.3 TSP algoritmusok általános fajtái	7
1.1.4 Az egyes korok teljes megoldású algoritmusainak eredményei	7
1.1.5 Egy változat, a Decay módszer	8
1.1.6 A Decay – Vogel módszer	9
1.1.7 A két stratégia statisztikája:	10
1.1.8 Egy harmadik változat, a legközelebbi szomszéd heurisztika	10
1.2 Ládarakás	11
1.2.1 Specifikáció:	11
1.3 Ládapakolás	11
1.3.1 Specifikáció:	11
1.3.2 Bevezető a változatokhoz	11
1.3.3 Ládapakolás változatai I. láda és vektorfedés	12
1.3.4 Ládapakolás változatai II. Online ládapakolás	14

Szállítmányozó és raktározó céget irányító és támogató program

1.3.5	Ládapakolás változatai III. Az NF algoritmus, helykorlátos algoritmusok	15
2	A programban használt technikák, módszerek	16
2.1	Dinamikus programozás	16
2.2	Elágazás-korlátozás módszere (branch and bound)	17
2.3	Heurisztikus megoldás	17
3	A java nyelvről	18
3.1	Általában	18
3.1.1	Röviden az előzményekről	18
3.1.2	Egyszerű	19
3.1.3	Objektumorientált	20
3.1.4		20
3.1.5	Architektúrafüggetlen és hordozható	20
3.1.6	Interpretált és dinamikus	21
3.1.7	Robosztus és biztonságos	23
3.1.8	Többszálú	24
3.1.9	Összefoglalás	25
3.1.10	Végezetül néhány kérdés, kétely	26
3.2	Néhány főbb specialitás a javaban	27
3.2.1	A program szerkezete	27
3.2.2	Változók és értékek	28
3.2.3	A főprogram	30
3.2.4	Vezérlési szerkezetek	31
3.2.5	Fordítás, futtatás	31
3.2.6	Absztrakt adattípusok létrehozása	32
3.2.7	Objektumok	32
3.2.8	Osztályok definíciója	33
3.2.9	Egyedváltozók	34
3.2.10	Osztályok leszármaztatása	34
3.3	A grafikus interfész	35
3.3.1	Komponensek és tulajdonosi hierarchiájuk	36
4	Az elkészült Program ismertetése	37
4.1	Amiket a grafikus interfészből használtam	37
4.2	TSP modulom	39
4.2.1	Bevezető	39
4.2.2	Összehasonlítás más algoritmusokkal	40
4.2.3	Egy program bemutatása a TSP - re	41
4.2.4	Összehasonlítása a modulomnak a fenti programmal.	43
4.3	Ládarakás	44
4.3.1	Specifikáció:	44
4.3.2	Hatékonyság:	44
4.3.3	Stratégiam	45
4.4	Ládapakolás	46
4.4.1	NP – nehéz megoldások	46
4.4.2	Stratégiam, összehasonlítás	47
5	A programom hatékonysága	48
6	Kitekintés	49
	Irodalomjegyzék:	50

1 Általános megoldási módok

1.1 TSP

Az utazó ügynök problémája egy kombinatorikus optimalizálási probléma. Kiváló példa a bonyolultság-elmélet által NP - nehéznek nevezett problémaosztályra. Az utazó ügynök problémához kapcsolódó matematikai feladatokkal először Sir William Rowan Hamilton és Thomas Penyngton Kirkman foglalkoztak az 1800-as években. Az utazó ügynök probléma általános változatát először az 1930-as években vizsgálták Bécsben, illetve a Harvardon, kiváltképp Karl Menger. A problémával később Hassler Whitney és Merrill M. Flood is komolyan foglalkozott a Princetonon.

A probléma:

Adva van n város, illetve az útiköltség bármely két város között, keressük a legolcsóbb utat egy adott városból indulva, amely minden várost pontosan egyszer érint, majd a kiindulási városba ér vissza.

$$\frac{(n-1)!}{2}$$
 út közül kell választanunk, ez ugyanis a Hamilton - körök száma az n pontú teljes gráfban.

Megjegyzés: Ez a képlet csak $n > 2$ esetén működik.

1.1.1 Kapcsolódó problémák

1.Ekvivalens gráfelméleti megfogalmazása a problémának: Adott teljes élsúlyozott gráf esetén keressük a legkisebb összsúllyal rendelkező Hamilton - kört. Megmutatható, hogy a kiindulási városba való visszatérés megkövetelése nem nehezíti a probléma számítási nehézségén, tehát minimális súlyú Hamilton - út keresése egy adott pontból is NP - teljes.

2.A probléma egy másik változata, amikor nem a legkisebb súlyú Hamilton - kört keressük, hanem azt, amelyikben a „legnehezebb” él súlya a lehető legkisebb. A logisztikai problémákon túl nagy gyakorlati jelentőséggel bír például a nyomtatott áramkörök gyártása során fúrórobotok ideális mozgásának megtervezésében.

1.1.2 Számítási nehézség

A legkézenfekvőbb megoldás az összes permutáció végignézése, és a legkisebb súlyú kiválasztása lenne, de tekintve, hogy ez $n!$ darab permutációt jelent (ahol n a városok száma), nagyobb n - ekre ez a megoldás kivitelezhetetlen. Dinamikus programozási technikák segítségével a probléma megoldásának lépésszáma felülről becsülhető $n^2 2^n$ - nel. Ez még exponenciális függvénye n - nek, de sokkal jobb, mint az $n!$ lépést használó „nyers erő” módszer.

1.1.3 TSP algoritmusok általános fajtái

Mivel tehát hatékony megoldás nem ismert a probléma megoldására sok város mellett, a következő célú algoritmusok a gyakoriak:

- A legjobb megoldást megkereső algoritmus, mely csak kisebb mennyiségű pontszám mellett hatékony.
- „Szuboptimális” vagy heurisztikus algoritmusok, melyek nagy valószínűséggel az optimális megoldáshoz jól közelítő megoldást adnak.
- A probléma speciális eseteivel foglalkozó, azt valamilyen megszorítás mellett hatékonyan megoldó algoritmusok.

1.1.4 Az egyes korok teljes megoldású algoritmusainak eredményei

Az idő múlásával az egyre finomodó technikáknak, és a számítástechnika fejlődésének köszönhetően egyre nagyobb mennyiségű városra sikerült megoldani a problémát:

- 1954: 49 Amerikai Egyesült Államokbeli város - G. B. Dantzig, D. R. Fulkerson, és S. Johnson.
- 1977: 120 város Németország környékéről - M. Grötschel.

- 1987: 2392 pont - M. W. Padberg és G. Rinaldi.
- 2004: Svédország 24978 városa - D. Applegate, R. Bixby, V. Chvátal, W. Cook, és K. Helsgaun.
- 2006: 85900 pontú probléma megoldása a CONCORDE algoritmus segítségével - D. Applegate, R. Bixby, V. Chvátal, W. Cook, D. Espinoza, M. Goycoolea és K. Helsgaun. Y [1]

1.1.5 Egy változat, a Decay módszer

És most lássunk néhány heurisztikán alapuló megoldást, amelyeket az a program használ, amit később összehasonlítok az én programom felhasználói felületével. Az elkövetkező leírásokkal azt kívánom érzékeltetni, hogy miért csökken egy egyszerűbb, kevésbé összetett algoritmus erőforrás igénye és ezáltal pontossága, nem úgy, mint az én megoldásom, ami egy legjobb megoldást szolgáltat, de nem megoldható kereteken belül, vagyis exponenciális futási idővel.

Ötletes okoskodások ezek a régmúlt emberétől. Igen, valahol ez lesz majd a fény az alagút végén: egy egyszerű de mégis frappáns megoldás, ami felfedezésekor majd minden szakértő a fejéhez kap, de ugyanakkor elismerően fog gondolni rá, ha elég jó lesz.

Dinamikus programozással, összefüggésében, minden részletre kiterjedően oldjuk meg a problémát, aminek egy optimális megoldás a jutalma, míg a heurisztikus programozás csak kiragad egy döntőnek látszó tulajdonságot, szerkezetet, szerkezet részletet a rendszerből és így próbál minél jobban közelíteni.

Lehet, hogy az a bizonyos fény az alagút végén a genetikus algoritmusok (lásd Kitekintés) lesznek.

Lássuk a példákat, a Decay módszer, a Decay – Vogel módszer és a legközelebbi szomszéd heurisztika. Először az alfejezet címében szereplőt:

Szállítmányozó és raktározó céget irányító és támogató program

Az algoritmus szerint lehetőleg a legnagyobb értékű költség elemeket jelentő viszonylatokat törölni kell a távolságmátrixból, hogy ne kerülhessenek beépítésre a körútba. A módszer főbb lépései:

1. Kiinduló adatok (távolságmátrix) meghatározása.

2. A távolságmátrix legnagyobb elemének megkeresése és törlése. Ha több ilyen elem van, akkor a törlés sorrendje tetszőleges lehet. A lépést addig kell ismételni, míg elő nem áll egy olyan helyzet, hogy a mátrix valamely sorában vagy oszlopában már csak egyetlen elem marad. Ha több olyan sor vagy oszlop maradt, melyben egy elem van, akkor a kisebb értékűt célszerű választani.

3. A megmaradt elem által meghatározott viszonylat része kell, hogy legyen a körútnak. Mivel az elem a viszonylat része lett, ezért törölni kell:

- az elemhez tartozó sor és oszlop összes többi (eddig nem törölt) elemét;
- a kiválasztott elemnek a mátrix főátlójára szimmetrikus párját;
- minden olyan elemet, amely ellentmond annak a feltételnek, hogy egy pontból csak egyetlen másik pontba vezethet körút.
- ha az utoljára kiválasztott viszonylattal nem sikerült olyan körutat létrehozni, amely minden pontot tartalmaz, akkor a 2. lépéshez vissza kell térni.

1.1.6 A Decay – Vogel módszer

A módszer a Dacey - módszer -ből származik.

1. Kiinduló adatok (távolságmátrix) meghatározása.

2. Valamennyi sorra és oszlopra meghatározzuk a két legkisebb elem értékét, és különbségüket a mátrix jobb oldalán illetve alul tüntetjük fel.

3. A maximális differenciájú sorban vagy oszlopban található legkisebb elem lesz a körút viszonylata.

4. A kiválasztott elem miatt törölni kell Dacey - módszernél ismertetett feltételeknek megfelelő elemeket.

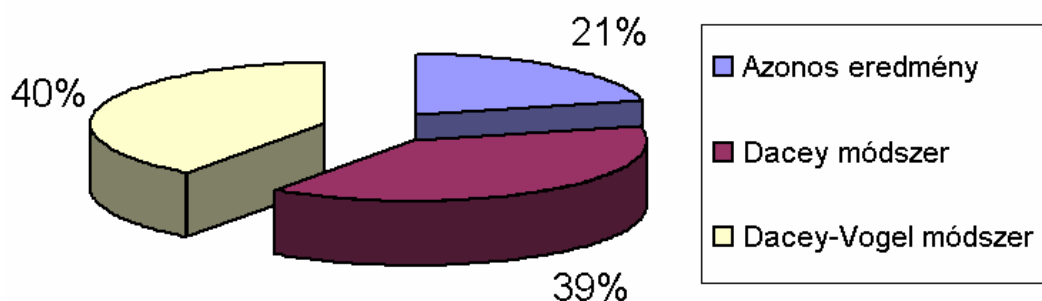
5. A törölt elemeket figyelmen kívül hagyva a 2. pont szerint újra meg kell határozni a sor és oszlop differenciák értékét, majd a legnagyobb differenciájú sorban vagy oszlopban található legkisebb eleme lesz a viszonylat része. A ciklust addig kell folytatni, amíg a körút teljes nem lesz.

1.1.7 A két stratégia statisztikája:

A tesztprogram eredményei:

- □ 2105 esetben (21,05%) azonos eredményt adott mindkét módszer;
- □ 3895 esetben (38,95%) a Dacey - módszer adott jobb eredményt;
- □ 4000 esetben (40%) a Dacey – Vogel - módszer bizonyult jobbnak.

A Dacey és a Dacey-Vogel algoritmus összehasonlítása szimmetrikus mátrixok esetén



1. táblázat: A Dacey és a Dacey - Vogel algoritmus összehasonlítása szimmetrikus mátrixok esetén

1.1.8 Egy harmadik változat, a legközelebbi szomszéd heurisztika

A következő algoritmust kapjuk:

1. Válasszunk tetszőlegesen egy várost.
2. Vizsgáljuk meg az összes, még ki nem választott várost, és válasszuk azt, amelyik a legközelebb van az aktuális városunkhoz.
3. Addig ismételjük az eljárást, amíg minden várost meg nem látogattunk.

Ez az algoritmus megközelítőleg N^2 idejű, ami jelentős javulást jelent az $N!$ – hoz képest. [2.1]

1.2 Ládarakás

1.2.1 Specifikáció:

Egy konténer raktárban, egy sorban, egymás mellett tárolnak n darab konténert. A raktáros át akarja rendezni a konténereket úgy, hogy néhányat egymásra rak. Az átrendezeit bizonyos konténerek egyesével történő átrakásával lehet végezni. Ha az átrendezeit során az i - dik konténerhelyen lévő konténert a j - dik konténerhelyre rakja át, ennek költsége $s_i - j$, ha a konténer súlya s_i . A raktáros olyan átrendezeit akar, amely után pontosan k konténerhelyen lesz konténer (esetleg egymásra rakva). A raktáros az optimális átrendezeit keresi, tehát amelyre az összköltség minimális. Ez a modul kiszámítja a raktár optimális átrendezeitének költségét, a továbbiakban ezt a tulajdonságot vizsgálom, de az optimális rakások helyeit is meghatározza!

1.3 Ládapakolás

1.3.1 Specifikáció:

Tegyük fel, hogy adott egy n tárgyból álló halmaz, ahol az i - dik tárgy s_i méretére $0 \leq s_i \leq 1$ egységnyi. Az összes tárgyat szeretnénk bepakolni minimális számú 1 egységnyi méretű ládába. Minden ládába a tárgyak bármely részhalmaza belefér, amelynek összmérete nem haladja meg az 1 egységet. Ebben a panelben azt adom meg a tárgyakról, hogy minimálisan hány ilyen ládába férnének bele. Ez a programmodul is hasznos lehet egy szállítványozó vállalat számára.

1.3.2 Bevezető a változatokhoz

A következő, néhány alfejezet erejéig lássunk, ízelítőül néhány egyszerű változatot és megoldást a ládapakolás problémára. Mindegyikük jól összeszedett és

világos, egyszerű, célravezető heurisztikák, csak úgy, amit én használtam (lásd később). Tanulságosak, híven tükrözik a sokféleségüket, egyszerűségüket és – nem elhanyagolhatóan, sokszor nagy örömeinkre – hatékonyságukat. Megvizsgálom aszimptotikus versenyképességi hányadosukat, amely mérőszám az algoritmus viselkedését jellemzi, ha az optimális megoldás a végtelenbe tart, majd megadom az összefoglaló végén a saját változatom aszimptotikus versenyképesség hányadosát is.

A probléma és változatai az NP – teljes osztályban vannak, vagyis nem oldhatóak meg polinomiális időn belül. A heurisztikus megoldások ismertetése után meg is adok ilyen exponenciális futási idejű megoldásokat.

1.3.3 Ládapakolás változatai I. láda és vektorfedés

A ládafedés feladatát szokás duális ládapakolási feladatnak is nevezni. A cél lefedni a maximális számú ládát, ahol egy ládát akkor fedünk le, ha a bele pakolt elemek összmérete legalább 1. A probléma online változatában a tárgyak egyenként jönnek, és a további tárgyakra vonatkozó információk nélkül kell elpakolnunk őket. Az online probléma megoldására a következő algoritmus került kifejlesztésre. DNF (dual next fitt) algoritmus:

Ha az aktuális láda még nincs lefedve, rakjuk bele a tárgyat. Egyébként nyissunk új ládát, és legyen az az aktuális láda.

Mivel minden ládára a benne levő tárgyak összmérete legfeljebb 2, továbbá egy láda lehet, ami nincs lefedve, és az optimális megoldásban sem lehet több ládát lefedni, mint amennyi a tárgyak összmérete, ezért adódik, hogy a DNF algoritmus aszimptotikusan $\frac{1}{2}$ - versenyképes. Szintén igazolást nyert az az állítás is, hogy nincs olyan online algoritmus amelynek az aszimptotikus versenyképességi hányadosa nagyobb, mint $\frac{1}{2}$.

A továbbiakban a ládafedési feladat egy általánosítását, a vektorfedést vizsgáljuk.

A tárgyak d - dimenziós vektorok. Egy láda akkor van lefedve, ha a benne levő tárgyak összmérete minden komponensben legalább 1. Az online problémára egy igen bonyolult algoritmust fejlesztettek ki, amelyben a paraméterek beállíthatóak úgy, hogy az aszimptotikus versenyképességi hányados tetszőlegesen közel legyen $1/(2d)$ - hez.

Szállítmányozó és raktározó céget irányító és támogató program

Mi a továbbiakban egy egyszerű lineáris idejű off - line $1/2$ aszimptotikus hányadossal rendelkező algoritmust mutatunk be a $d = 2$ esetre.

Az algoritmushoz az elemeket a következő csoportokba osztjuk. Egy elem $(-, -)$ típusú, ha mindkét koordinátája legfeljebb $1/2$. Egy elem $(-, +)$ illetve $(+, -)$ típusú, ha az első komponense legfeljebb $1/2$, a második komponense $1/2$ és 1 között van, illetve ha az első komponense $1/2$ és 1 között van a második komponense legfeljebb $1/2$. Végül egy elem $(+, +)$ típusú, ha mindkét koordinátája nagyobb, mint $1/2$ és legfeljebb 1 . Az algoritmus a következő lépéseket hajtja végre.

1. lépés Mindaddig, amíg van legalább kettő $(-, -)$ típusú elem, azokat vonjuk össze egy új elemmé, amelynek a koordinátái az eredeti koordináták összegei. Az új elem bármelyik típusba eshet, de mindkét koordinátája legfeljebb 1 . Ha egy darab $(-, -)$ elem marad dobjuk el.

2. lépés Mindaddig, amíg van két - két $(-, +)$ és $(+, -)$ típusú elem vonjunk össze ezekből egyet - egyet egy párba. Ha az összevont elem mindkét koordinátája legfeljebb 1 , akkor helyettesítsük őket az összevont $(+, +)$ típusú elemmel. Ha mindkét koordináta legalább 1 , akkor fedjük le egy ládát a két elemmel. Egyébként az egyik koordináta 1 és $3/2$ között van, a másik $1/2$ és 1 között. Feltehetjük, hogy az első a nagyobb, a másik eset szimmetrikus. Ekkor egy $(-, +)$ elemmel letudjuk fedni a két elemet tartalmazó ládát. Ha valamelyik típusból egy marad, akkor azt dobjuk el. Tegyük fel, hogy $(-, +)$ típusú elemek maradtak a másik eset szimmetrikus.

3. lépés A $(+, +)$ elemeket rakjuk páronként ládába, minden pár lefed egy ládát. Ha egy elem maradt azt dobjuk el.

4. lépés A megmaradt $(-, +)$ típusú elemekkel fedjük le a ládákat a DNF algoritmust használva az első koordináta szerint. A maradék láda tartalmát, amit már nem tudtunk lefedni eldobjuk. A fentiekben bemutatott algoritmusra teljesül a következő állítás.

Tétel. Az algoritmus aszimptotikusan $1/2$ - approximációs.

Bizonyítás: A (2) és (3) lépésben lefedett ládák mindegyikében mindkét komponensben legfeljebb 2 az összméret. Továbbá a (4) - es lépésben lefedett ládáknak az összméret az első komponensben legfeljebb 2 . (Ha a szimmetrikus eset állna fenn, akkor a második komponensben lenne legfeljebb 2 az összméret.)

Tehát az első komponensek összege az összes tárgyra nézve legfeljebb $2A(I)+C$ ahol C az eldobott konstans mennyiségű tárgyak első komponenseinek összege.

Másrészt ez az összeg egy felső korlát az $OPT(I)$ értékre, amiből következik a tétel állítása. [3]

1.3.4 Ládapakolás változatai II. Online ládapakolás

Ebben a részben az online ládapakolási problémát vizsgáljuk, amely problémában az algoritmusnak az i - edik tárgyat az $a_1, \dots, a_i$ értékek alapján kell elhelyezni a további tárgyra vonatkozó információk ismerete nélkül. Az online algoritmusok hatékonyságát elméleti szempontból két módszerrel szokás vizsgálni.

Az átlagos eset analízis során feltételezünk valamilyen típusú eloszlást a tárgyak méretéről és a használt ládák várható számát vizsgáljuk. A másik megközelítés a versenyképességi analízis, amely egy legrosszabb esetkorlát. Ebben a részben ez utóbbi módszert használjuk, ezért az alábbiakban megadom a pontos definíciót.

A ládapakolási problémák elemzésére az aszimptotikus versenyképességi hányadost vizsgálják. Egy A algoritmus aszimptotikus versenyképességi hányadosa R_A^∞ a következő formulákkal definiálható:

$$R_A^n = \max\{A(L)/OPT(L) \mid OPT(L) = n\}$$

$$R_A^\infty = n \xrightarrow{\limsup} \infty R_A^n.$$

Az aszimptotikus hányados fő tulajdonsága az, hogy azt vizsgálja, miként viselkedik az algoritmus akkor, ha az input mérete nagy, azaz ha az optimális költség végtelenhez tart. Ez azt jelenti, hogy az algoritmus szabadon helyezheti el a kezdeti elemeket a listáról.

A fentiekben definiált ládapakolási problémának számos változata, általánosítása van. Az alábbiakban megemlítek további néhányat. A ládapakolási probléma három különböző módon általánosítható több dimenzióra. Az első általánosítás a vektorpakolás, amelyben a tárgyak d - dimenziós vektorok, és úgy kell elpakolni őket, hogy minden ládában minden koordinátára az ott szereplő értékek összege ne legyen nagyobb 1 - nél. A második általánosítás a dobozpakolás, ahol többdimenziós dobozokat kell pakolni többdimenziós egységládákba.

Végül a harmadik általánosítás a sávpakolás, ahol egy egységnyi széles sávba pakolunk tárgyakat és célunk, minimalizálni a használt magasságot.

1.3.5 Ládapakolás változatai III. Az NF algoritmus, helykorlátos algoritmusok

Érdeemes még megemlítenünk azt a modellt, amelyben az egy időben nyitott ládák száma korlátozott. Ez azt jelenti, hogy amennyiben a nyitott ládák száma elér egy k - korlátot, akkor ezt követően csak akkor nyithatunk új ládát, ha az eddig nyitott ládák valamelyikét bezárjuk, ami azt jelenti, hogy többet nem használhatjuk.

Ha a nyitott ládák száma csak egy lehet, akkor az egyetlen algoritmus, amely használható a következő NF algoritmus: Amennyiben a tárgy elfér a nyitott ládában, tesszük oda! Ellenkező esetben zárjuk be a nyitott ládát, nyissunk egy új ládát és tesszük abba a tárgyat!

Tétel. Az NF algoritmus aszimptotikus versenyképességi hányadosa 2.

Bizonyítás: vegyünk egy tetszőleges σ tárgysorozatot. Jelölje n az NF algoritmus által használt ládák számát, továbbá legyen S_i , $i = 1, \dots, n$ az i - dik ládában levő tárgyak méreteinek összege. Ekkor $S_i + S_{i+1} \geq 1$, hiszen ellenkező esetben az $i + 1$ -edik láda első eleme elfért volna az i - dik ládában, ami ellentmond az algoritmus definíciójának. Következésképp a tárgyak méreteinek összege legalább $\left\lfloor \frac{n}{2} \right\rfloor$. Másrészt az optimális offline algoritmus sem rakhat 1 - nél több összméretű tárgyakat egy ládába, így azt kapjuk, hogy $\text{OPT}(\sigma) \geq \left\lfloor \frac{n}{2} \right\rfloor$.

Ez azt jelenti, hogy

$$\frac{NF(\sigma)}{\text{OPT}(\sigma)} \leq \frac{n}{\left\lfloor \frac{n}{2} \right\rfloor} \leq 2 + 1/\left\lfloor \frac{n}{2} \right\rfloor.$$

Másrészt ha n tart végtelenbe, akkor $1/\left\lfloor \frac{n}{2} \right\rfloor$ tart a 0-hoz, amivel igazoltuk hogy az algoritmus aszimptotikusan 2 - versenyképes.

Most megmutatjuk, hogy az algoritmus aszimptotikus versenyképességi hányadosa legalább 2. Ehhez tekintsük minden n - re a következő σ_n sorozatot.

A sorozat $4n$ tárgyból áll, a $2i - 1$ - dik tárgy mérete $1/2$, a $2i$ - dik tárgy mérete $1/2n$, ahol $i = 1, \dots, 2n$. Ekkor az NF algoritmus az i - dik ládába a $2i - 1$ - dik és a $2i$ - dik tárgyat teszi, és $NF(\sigma_n) = 2n$. Az optimális algoritmus az $1/2$ méretű tárgyakat

párosítja, és a kis tárgyakat egy további ládába teszi, így $OPT(\sigma_n) = n + 1$. Mivel $NF(\sigma_n)/OPT(\sigma_n) = 2 - 2/(n + 1)$ a 2 értékhez konvergál, ha n tart végtelenbe, ezért igazoltuk, hogy az algoritmus aszimptotikus versenyképességi hányadosa legalább 2.

Itt érdemes megemlítenünk, hogy amennyiben egynél több (de korlátozott számú) láda lehet nyitva, az NF algoritmusnál jobb algoritmusok is ismertek. A jelenlegi legjobb algoritmusok a harmonikus algoritmusok családjába tartoznak, ahol az alapötlet az, hogy a $(0, 1]$ intervallumot részintervallumokra osztjuk, és minden tárgynak az az intervallum lesz a típusa, amely intervallumba a mérete esik. A különböző típusú tárgyakat különböző ládába pakoljuk, az algoritmus párhuzamosan alkalmaz egy-egy NF algoritmust az egyes típusokhoz tartozó tárgyakra. [4]

2 A programban használt technikák, módszerek

- Dinamikus programozás (1. Tsp (Branch and Bound), 2. Láda rakás)
- Heurisztikus megoldás (3. Láda pakolás)

2.1 Dinamikus programozás

A dinamikus programozás éppúgy, mint az oszd – meg – és - uralkodj módszer, a feladatot részfeladatokra való osztással oldja meg. (A „programozás” szó ebben az összefüggésben egy táblázatos módszerre utal, nem egy számítógépes program írására.) Az oszd – meg – és - uralkodj módszer felosztja a feladatot független részfeladatokra, melyeket ismételten megold, és a részfeladatok megoldásait az eredeti feladat megoldása céljából egyesíti. Ezzel szemben a dinamikus programozás akkor alkalmazható, ha a részproblémák nem függetlenek, azaz közös részproblémáik vannak. Ilyen helyzetben az oszd – meg – és - uralkodj módszer a szükségesnél többet dolgozik, mert ismételten megoldja a részproblémák közös részproblémáit. A dinamikus programozás minden egyes részfeladatot és annak minden részfeladatát pontosan egyszer oldja meg, az eredményt egy táblázatban tárolja, és ezáltal elkerüli az ismételt számítást, ha a részfeladat megint felmerül.

A dinamikus programozást optimalizálási feladatok megoldására használjuk. Ilyen feladatoknak sok megengedett megoldása lehet. Mindegyiknek van értéke, és mi

az optimális (minimális vagy maximális) értékűt kívánjuk megtalálni. Egy ilyen megoldást „egy optimális megoldás” - nak nevezzük, nem pedig az optimális megoldásnak, mivel más megoldások is elérhetik az optimális értéket.

Egy dinamikus programozási algoritmus kifejlesztése négy lépésre bontható.

1. Jellemezzük az optimális megoldás szerkezetét.
2. Rekurzív módon definiáljuk az optimális megoldás értékét.
3. Kiszámoljuk az optimális megoldás értékét alulról felfelé történő módon.
4. A kiszámított információk alapján megszerkesztünk egy optimális megoldást.

Az 1 - 3. lépések jelentik az alapját annak, ahogy a dinamikus programozás meg tud oldani egy feladatot. A 4. lépés elhagyható, ha csak az optimális értékre vagyunk kíváncsiak. Ha végre kell hajtanunk a 4. lépést, akkor a 3. lépésben gyakran olyan kiegészítő információkat határozzunk meg, melyek megkönnyítik az optimális megoldás előállítását. [5]

2.2 Elágazás-korlátozás módszere (branch and bound)

Ha a megoldáskezdemények tere fa, akkor a probléma egy, vagy az összes megoldását kereshetjük a fák szint szerinti bejárásának megfelelő stratégiával. A stratégiát még általánosabban használhatjuk, ugyanis az aktuális pontot tetszőlegesen választhatjuk az aktív pontok közül. A lényeg, hogy a választott aktuális pont minden fiát kigeneráljuk, és ha lehetséges megoldás, akkor betesszük az aktív pontok halmazába. Tehát az algoritmus egy adagolót használ az aktív pontok tárolására. [6]

2.3 Heurisztikus megoldás

Hogy hatékonyan oldjunk meg nehéz problémákat, gyakran kompromisszumokat kell kötnünk, és olyan eljárásokat kell alkalmaznunk, amelyek már nem garantálják a legjobb választ, de mindig nagyon jól megközelítik azt. Ehhez bevezetjük a heurisztika fogalmát. A heurisztika egy olyan technika, amely javítja a kereső algoritmus hatékonyságát, feláldozva akár a teljesség igényét is. Egyes heurisztikák nem bántják a

teljességet, míg mások (a legjobbak közül sok) elveszíthetnek egy kitűnő megoldást. De átlagban javítanak az algoritmusokon. Heurisztikát alkalmazva remélhetjük, hogy jó megoldást (bár valószínűleg nem az optimálisat) kapunk kevesebb, mint exponenciális idő alatt. [7]

3 A java nyelvről



3.1 Általában

3.1.1 Röviden az előzményekről

Az 1990-es évek elején valahol a SUN berkeiben elindult egy kevésbé fontos projekt azzal a céllal, hogy a cég betörjön a felhasználói elektronikai piac egy új, az ún. "okos", processzorral vezérelt, programozható (*smart*) készülékeket alkalmazó területére. E készülékcsalád jellegzetes képviselője a kábel - TV társaságok által használt vezérlő (*set - top box*). Az ilyen készülékek programozásához igény volt olyan architektúra - független technológiára, amely lehetővé tette a kész programok a hálózaton keresztül a készülékbe letöltését, megbízható futtatását.

A projekt kezdetben a C++ nyelvet használta, ám a fejlesztők ezt is, a többi, akkor hozzáférhető programozási nyelvet is alkalmatlannak találták a célkitűzéseik maradéktalan megvalósítására, hát új nyelvet terveztek maguknak. Kiindulási alapként a C++ nyelvet használták, kigyomlálva belőle - az általuk - bonyolultnak, nem megbízhatónak talált szerkezeteket, hozzáadva innen-onnan átvett, hasznosnak tűnő ötleteket, nyelvi elemeket.

A felhasználói elektronikai alkalmazások lassabban fejlődtek, mint azt előre várták, a projekt szép csendben el is halt volna, ha közben az Internet hálózat nem indul rohamos fejlődésnek. Szerencsére észrevették, hogy az Internet hálózat hasonló körülményeket teremt és hasonló igényeket támaszt egy új programozási technológiával szemben.

3.1.2 Egyszerű

A nyelv szintaxisa és szemantikája nagyban hasonlít a sokak által ismert C illetve C++ programozási nyelvhez, megkönnyítve a kezdeti ismerkedést. A C++ nyelvből elhagytak néhány - programozói vagy fordítóprogram írói szempontból - bonyolult elemet. Kimaradt például az operátorok felüldefiniálásának lehetősége (*operator overloading*), a többszörös öröklődés. Eltűnt a *goto* utasítás, az automatikus típuskonverziók (*coercion*), az összetett adatszerkezetek közül az *union*, illetve C++ - ban már amúgy is szükségtelen *struct*.

Azért szerencsére nem gyomláltak ki mindent a C++ - ból, megmaradt például a futásidejű hibák lekezelésének mechanizmusa, az. ún. kivételkezelés (*exception handling*). Bár az "igazi programozók" megkönnyezik az eltávozott *goto* utasítást, de helyette címkézett ciklusokat, többszintű *break* és *continue* utasítást kaptunk.

Sokaknak elsőre - többeknek talán másodszorra is - furcsa, de a Jáva nem használ mutatókat (*pointer*), egy csapásra megszüntetve ezzel a C programozók kedvelt programozási hibáinak egész seregét. A programozó munkáját nagymértékben megkönnyíti az is, hogy a nyelv automatikusan felszabadítja a már nem használt tárterületeket (szemétgyűjtés, *garbage collection*).

Ízelítőül álljon itt a kötelező "Szervusz világ" program egy kicsit módosított változata, amely az első paraméterben megadott szöveget írja ki a "World" helyett, már amennyiben van ilyen.

```
class HelloWorldApp
{
    public static void main (String args[])
    {
        System.out.println("Hello ");
        if (args.length == 0)
            System.out.println("World!");
        else
            System.out.println(args[0] + "!");
    }
}
```

3.1.3 Objektumorientált

Manapság az objektumorientáltság divatos programozási paradigma, bár a szakirodalom nem teljesen egységes a kritériumainak meghatározásában. A Jáva lényegében a C++ objektumorientált tulajdonságait tartalmazza. A programozó absztrakt adattípusként viselkedő osztályokat definiálhat, az osztályok műveleteket - módszereket - tartalmaznak, amelyek a rejtett adatrepresentáción (egyedváltozók) operálnak. Létrehozhatunk objektumokat, azaz egyes osztályokba tartozó egyedeket. Osztályok definiálásánál felhasználhatunk már meglévő osztályokat, az új osztály (a leszármazott) örökli a szülő adatait, módszereit. A módszerek hívásánál a meghívott módszer futási időben, objektum aktuális típusának megfelelően kerül kiválasztásra (virtuális módszerek, polimorfizmus).

Az egyes osztályokban definiált változók és módszerek láthatóságát a C++ - ban megismert módon - *private*, *protected* és *public* - lehet megadni.

Eltérést jelent a C++ - hoz képest, hogy a Jávában a beépített, egyszerű adattípusú - numerikus, logikai és karakter típus - változók kivételével minden objektum; az egyetlen összetett adattípus, a tömb (*array*) teljes értékű osztályként viselkedik. A program nem tartalmaz globális változókat és globális eljárásokat, minden adat és eljárás valamilyen objektumhoz, esetleg osztályhoz kötődik. Persze a C++ - t ismerők tudják, hogy a globális változókat és függvényeket helyettesíteni lehet ún. osztályváltozókkal és statikus függvényekkel, ezek itt is használhatók.

A Jávában minden módszerhívás - a fent említett statikus módszerek kivételével - virtuális. A C++ - hoz hasonlóan lehetőségünk van az egyes objektumok típusát futási időben lekérdezni. Az osztályok mellett a Jáva az Objective - C programozási nyelvből átvette az *Interface* fogalmat. Az *Interface* nem más, mint módszerek egy halmaza - adatszerkezeteket, egyedváltozókat nem tartalmaz -, amelyet egyes osztályok megvalósíthatnak (implementálhatnak). A Jáva a C++ - val ellentétben nem engedi meg a többszörös öröklődést, viszont *Interface* - ek használatával, egyszerűbben, kevesebb implementációs problémával hasonló hatást lehet elérni.

3.1.4

3.1.5 Architektúrafüggetlen és hordozható

Napjaink hálózatait heterogén hardver- és szoftver architektúrájú számítógépek alkotják. A programok fejlesztését nagymértékben megkönnyítené, ha a forráskódból előállított program bármely architektúrán, azonos módon futna. A cél elérése végett a Jáva nyelv nem tartalmaz architektúra- vagy implementációfüggő elemeket.

Ahhoz, hogy a lefordított program változtatás nélkül futtatható legyen különböző hardver architektúrákon, a fordítóprogram a programot nem egy konkrét processzor gépi kódjára, hanem egy képzeletbeli hardver - virtuális gép (*virtual machine*) - utasításrendszerére fordítja le. Az így létrejött közbülső, ún. Byte kódot töltjük le a célarchitektúrára, ahol a virtuális gépet megvalósító program értelmezi és hajtja végre.

A hordozhatóság nem csak a virtuális gépi utasítások, hanem a nyelv mellett szabványosított rendszerkönyvtárak szintjén is jelentkezik, ezek a könyvtárak valósítják meg a legfontosabb, operációs rendszerekhez kötődő feladatokat, mint például a be- és kiviteli rendszert, vagy a programok grafikus kezelői felületét.

Egy új architektúrán akkor futtathatók a Jáva programok, ha már implementálták rá a virtuális gépet, beleértve a rendszerkönyvtárakat is. A virtuális gépet C - ben írták.

3.1.6 Interpretált és dinamikus

Az interpretált végrehajtás - kombinálva a klasszikus kapcsolatszerkesztő (*linker*) helyett futás idejű betöltéssel - a fejlesztési ciklust nagymértékben felgyorsítja. A Jávából eltűntek a C -ből ismert header állományok, feltételes fordítás, más programállományok fordítás közbeni beolvasása (*#include*). A lefordított programok tartalmazzák a szerkesztéshez szükséges minden információt. Elég csak a megváltozott állományokat lefordítanunk - nincs szükség a C -nél megszokott *make* programra, a forrásállományok közötti függőségek feltérképezésére -, a program máris futtatható. Egyébként a Jáva támogatja a nagybani programozást, összetartozó osztályok egy csomagba (*package*) foghatók, egyszerre fordíthatók. A láthatóság is figyelembe veszi a csomagokat, a C++ explicit *friend* deklarációjának szerepét itt a csomagra - de csak a csomagra - vonatkozó láthatóság veszi át. A kapcsolatszerkesztő helyét az ún. osztály - betöltő (*class - loader*) veszi át, amely futás közben - ha szükség van rá - betölti az egyes osztályokat megvalósító lefordított, Byte kódú programállományokat. Az osztály - betöltő nem csak helyi állományokból, de szükség esetén, a hálózaton keresztül is képes kódot letölteni.

Szállítmányozó és raktározó céget irányító és támogató program

Többek között a betöltő feladatának megkönnyítése végett a Byte kód tartalmaz a forráskódból átvett szimbolikus - illetve típus információkat. Lássunk egy példát arra, hol jöhet jól ez az információ.

Az objektumorientált programozási paradigma egyik nagy ígérete, hogy általános célú, könnyen felhasználható osztály - könyvtárakat, "szoftver-IC-eket" hozhatunk létre a segítségével. Sajnos a C++ nyelv ilyen tekintetben nem váltotta be teljesen a hozzá fűzött reményeket. Nagyon nehéz olyan osztályokat tervezni, amelyeket később nem kell majd úgy módosítani, hogy ne kelljen azt például új - bár a programozók elől rejtett - adatkomponensekkel vagy módszerekkel bővíteni. Bár az osztály külső interfésze nem feltétlen változott meg, ám a C++ az osztály reprezentációját, tárbeli elrendezését kihasználó fordítási mechanizmusa miatt ilyenkor az összes, a megváltozott osztályt - akár csak öröklésen keresztül - felhasználó programot újra kell fordítani. Ezt hívják "törékeny alaposztály" (*fragile base class*) problémának. A Jáva ezt a problémát úgy kerüli meg, hogy az osztályok adatkomponenseire, módszereire a Byte kódban is szimbolikus hivatkoznak, a hivatkozások konkrét címekké kötése csak futási időben, a virtuális gépben történik meg.

A közbülső kódban megmaradt szimbolikus információk megkönnyítik a programok nyomkövetését. Sajnos az interpretált végrehajtásnak van egy nagy hátránya is, a programok sokkal - becslések szerint 10 – 20 - szor - lassabban futnak, mint a gépi kódú megfelelőik. Bár a virtuális gép elég ügyesen lett kitalálva és a fordítóprogram is mindent megtesz azért, hogy ezt a virtuális architektúrát a lehető legjobban kihasználja (pl. regiszter allokáció optimalizálása), de még így sem vetekedhet a gépi utasítások sebességével. Ehhez még hozzáadódik a szimbolikus hivatkozások feloldásának ideje, a különböző betöltési - és futásidejű ellenőrzések - ld. később -, a tárgyazdalkodási modellel járó szemétygyűjtési algoritmus futásához szükséges idő.

E lassúság ellen jelenleg nem sokat tehetünk, legfeljebb azzal vigasztalhatjuk magunkat, hogy az alkalmazások jelentős részénél - gyakori felhasználói közreműködést, vagy hálózati kommunikációt igénylő programoknál - a program sebessége nem a legfontosabb követelmény. Persze a Jáva programok meghívhatnak gépi kódú (*native*) eljárásokat is, de ezzel elvesztjük az architektúra - függetlenség, hálózaton letölthetőség tulajdonságát. Ugyanilyen korlátozásokkal használhatunk jelenleg fejlesztés alatt álló, béta változatú Java - C fordítóprogramot is, az így - egy C fordítóval - előállított program lényegesen gyorsabban fut majd.

Legigéretesebbnek az úgynevezett "röptében fordítás" (*just – in - time, on – the - fly compilation*) ötlete tűnik. A Byte kód úgy tele van tömve szimbolikus információkkal, hogy elvileg nem nehéz - ezt kiindulási nyelvnek tekintve - optimalizáló fordítóprogramot írni hozzá. Ráadásul a virtuális gép utasításrendszere nagyon hasonlít napjaink processzoraiéhoz, könnyű belőle jó kódot generálni. A fordítás történhet az egyes osztályok betöltésekor - a szükséges ellenőrzések után -, esetleg végrehajtás közben. Futás közben esetleg az is eldőlhet, hogy érdemes-e az adott programrészletet lefordítani, mert gyakran használjuk, avagy megmaradhat interpretáltnak.

Másik érdekes kísérlet, hogy a SUN a Jáva virtuális gépet szilíciumon is megvalósítja, hamarosan kaphatók olyan mikroprocesszorok, amelyek a Jáva Byte kódot közvetlenül futtatják.

3.1.7 Robosztus és biztonság

Ez a két fogalom a Jáva esetében kéz a kézben jár. Robosztus egy nyelv, ha megakadályozza vagy futás közben kiszűri a programozási hibákat, biztonságos, ha megakadályozza, hogy rosszindulatú programok kerüljenek a rendszerünkbe. Mindkét célkitűzés eléréséhez gyakran hasonló módszereket használhatunk.

A robusztusság nyelvi szinten a szigorú, statikus típusosságban jelenik meg. Minden adatnak fordításkor jól definiált típusa van, nincsenek automatikus konverziók, az explicit konverzió csak kompatibilis típusoknál sikerül, egyébként legkésőbb futtatáskor programhibát (*exception*) okoz. A mutatók eltűnésével rengeteg potenciális hibalehetőség eltűnt a nyelvből - pl. NULL pointerhivatkozás, levegőben lógó mutatók -, persze elvesztettük az "igazi programozók" egyik kedvencének, az *int* -eknek és a mutatóknak ide - oda alakíthatóságának lehetőségét is. A dinamikus szeméthyűjtés megkímél bennünket a hasznos memória elszivárgásától (*memory leak*). Az egyedüli összetett adatszerkezet, a tömb használatakor a túlcímzést futási időben ellenőrzik. Az osztály - betöltő arra is figyel, hogy a módszereket megfelelő típusú paraméterekkel hívjuk meg.

A biztonság (*security*) a robusztussággal kezdődik, a fordító csak korrektül viselkedő programokat ad ki magából. Ez elegendő lehet önálló alkalmazásoknál, de a

programkák letöltésénél ennél többre van szükség. Kezdjük azzal, hogy ki garantálja, hogy a letöltött Byte kódot valóban egy megbízható Jáva fordító hozta létre, nem pedig egy bit - betyár barkácsolta össze a hexadecimális szerkesztőjével? A Byte kód minden utasítása információt tartalmaz az operandusok típusáról, az osztály - betöltő - függetlenül attól, hogy a helyi háttértárról vagy a hálózatról tölt be - ellenőrzi, hogy a program megfelel - e a nyelv szabályainak. Eldönti például, hogy minden operandus valóban a megfelelő típusú, nem használjuk ugyanazt az adatot más - más típusúként is. Ellenőrizhető az is, hogy a módszerek a vermiiket konzisztensen használják-e, valamint hogy a kód nem fér - e hozzá számára nem engedélyezett - a nyelv definíciója szerint láthatatlan, rejtett - adatkomponensekhez, módszerekhez.

A betöltő egy hivatkozott osztályt először mindig a helyi háttértárból próbál betölteni, csak akkor fordul a hálózaton elérhető kiszolgálóhoz, ha az osztály nincs meg a helyi rendszeren. Így elkerülhető, hogy trójai falóként valamelyik rendszerkönyvtár helyett azonos nevű, távolról betöltött programot futtassunk.

Ha a betöltött programkák átjutottak a betöltő konzisztencia - ellenőrzésén, akkor a virtuális gép felügyelete alatt kezdenek futni, ez ellenőrzi, hogy a programok csak engedélyezett tevékenységet hajtanak végre. Szigorúan szabályozott - vagy teljesen tiltott - például helyi állományokhoz való hozzáférés, tiltott más helyi programok indítása, jelentősen korlátozott a hálózaton felvehető kapcsolatok címzettje. Sajnos e biztonsági szabályok erősen korlátozzák a programkák képességeit. Biztonsági szempontból legfeljebb a helyi rendszerből betöltött - többé-kevésbé megbízható -, illetve hálózatról letöltött - eleve gyanús - osztályok között lehet különbséget tenni. Későbbiekben lehetőség lesz nyilvános kulcsú titkosítás segítségével azonosítható forrású, garantáltan eredeti, változatlan, "megbízható" programok letöltésére és futtatására is.

3.1.8 Többszálú

A programok jelentős része párhuzamosan végrehajtható részletekre - vezérlési szálakra - bontható. Így jobban kihasználható a számítógép központi egysége, a programok a külső - például felhasználói - eseményekre gyorsabban reagálhatnak. Az egymással kommunikáló, viszonylag laza kapcsolatban álló szálakra bontott feladat könnyebben áttekinthető, megvalósítható és belőhető. A többszálú programozáshoz a

Jáva nyelvi szinten biztosítja az automatikus kölcsönös kizárást: szinkronizált (*synchronized*) módszerek vagy utasítások, valamint a szálak létrehozásához, szinkronizációjának megvalósításához a rendszerkönyvtár tartalmaz egy ún. *Thread* osztályt.

3.1.9 Összefoglalás

A Jáva számos fent felsorolt tulajdonsága miatt érdekes, figyelemre méltó programozási nyelv; a nyelv tervezői sokat tanultak a korábbi tapasztalatokból. Bár valószínűleg a C++ - t, mint objektumorientált nyelvet a közeljövőben nem fogja kiszorítani, de bizonyos - nem túl nagyméretű, interaktív, grafikus felhasználói felületű - feladatok megoldásánál hódítani fog. Az pedig, hogy szorosan összefonódik a *World Wide Web* - el, garantálja a rohamos elterjedését, színesebbé, gazdagabbá téve az Internet hálózat világát. Bár nem kell feltétlenül hálózathoz kötődnie: nem lennék meglepve, ha hamarosan multimédiás CD - k lejátszásához is Jávával felcsinosított böngészőket használnánk.

Persze a Jáva képességeivel nem egyedülálló napjaink számítástechnikájában. Hasonló úton indult el az Apple a Dylan rendszerével, a Microsoft pedig a Blackbird - del, emlegetnek egy Python nyelvet, a Free Software Foundation is foglalkozik virtuális gépen alapuló program - interpretálással és még lehetne sorolni a hasonló célú nyelveket, rendszereket.

Ha a Jáva nyelv mégis elterjed, akkor ehhez hozzájárul a jó reklám, az ügyes marketing stratégia valamint a felhasználók és legfőképpen a szoftvergyártók erkölcsi és anyagi támogatása. Márpedig a szoftvergyártók tényleg ráharapni látszanak az új technológiára, olyannyira, hogy reménytelennek látszik teljes felsorolást adni azokról a cégekről, amelyek csatlakoztak, forrás licenzet vásároltak. Remélem, nem sértem meg a kimaradókat, a teljesség igénye nélkül: a SUN - nál külön részleget hoztak létre a Jáva fejlesztésekre. A WWW szempontjából talán a legjelentősebb esemény, hogy a Netscape megvásárolta a technológiát és integrálja az új böngészőjébe. Ha már a licenszekről van szó: nincs szükség licenszre ahhoz, hogy valaki Jáva programot illetve programkákat fejlesszen, a lefordított programot terjessze. Azon OEM szoftverfejlesztők, akik a Jáva technológiát integrálni akarják termékeikbe, forrás licenzet vásárolhatnak.

A Jáva technológiához kapcsolódóan csomó név és grafika a SUN - regisztrált - védjegye. Ezek közül a cikk elején látott Duke és a HotJava szimbóluma, a gözölgő kávéscsésze csak a SUN által használható. Hamarosan lesz egy ún. "Jávával turbósított" (*Java Powered*) grafika az olyan WWW lapokhoz, amelyek Jáva programkákat tartalmaznak, valamint egy "Jávával kompatibilis" (*Java Compatible*) grafika azon OEM termékeknek, amelyek előállítói megvásárolták a forrás licenszet, termékükbe beépítették ezt a technológiát - pl. új architektúrára implementálták a virtuális gépet - és sikeresen megfeleltek a - most készülő - kompatibilitási tesztnek.

Ha valaki Jáva programokat akar fejleszteni, legegyszerűbb, ha a SUN - től ingyenesen letölthető Jáva Fejlesztői Csomagot (*Java Development Kit*) használja, ami tartalmaz egy fordítóprogramot (*javac*), egy nyomkövetőt, hibakeresőt (*jdb*) a futtatáshoz szükséges virtuális gépet és egy a programkákat futtató alkalmazást (*AppletViewer*).

3.1.10 Végezetül néhány kérdés, kétely

A Jáva technológia hatalmas léptekkel fejlődik. Látszólag nagy a nyomás a fejlesztőgárdán, hogy minél hamarabb "végleges" változattal álljanak elő. A nagy sietségben maradhatnak a rendszerben átgondolatlan, elnagyolt megoldások. "Jó" - azaz rossz - példa erre a grafikus programozási könyvtár (*AWT, Abstract Windowing Toolkit*) esete, amelyet az alfa és béta verzió között teljesen átírtak: a fejlesztők beismerték, hogy a Netscape sürgette őket. Persze az objektumorientált könyvtárak bővíthetők, akár lecserélhetők is, de nem mindegy, mi kerül az 1.0 - s verzió specifikációba. "Ami tapad, az ragad", a korábbi verziókkal való kompatibilitás megőrzése miatt a rendszerek teherként hurcolják magukkal az első verziók elszett döntéseit.

Az sem látszik még, hogy a technológiát mennyire fogja egy - két erős cég a saját érdekeinek megfelelő irányába elvinni, avagy megmarad - e szélesebb körű érdekeket, véleményeket tükröző, nyílt technológiának. A C++ nyelvre is ráfért az elmúlt években bekövetkezett, széles felhasználói kört megmozgató tisztázási, szabványosítási eljárás, gondolom a Jávának sem ártana. Nem lehet látni például, hogy azon felül, hogy a Jáva

karakterei 16 bites Unicode rendszerben vannak ábrázolva, valaki foglalkozna a nemzetköziesítés (*internalization*) kérdéseivel.

Sokan attól is félnek, hogy a "végleges" verzió megjelenése után a SUN szigorít a licenz politikáján, túl költségessé teszi a fejlesztőrendszerét, vagy pl. a HotJava - t. A WWW hálózat terjedésében jelentős szerepet játszott az a tény, hogy a szükséges programok - kiszolgálók is, böngészők is - szabadon hozzáférhetők, ingyenesek voltak.

Érdekes lehet, hogy a felhasználók - főleg a nagy cégek - mennyire fogadják el a rendszerbe beépített biztonsági mechanizmusokat. Első ránézésre megbízhatónak - ha néha túl korlátozottnak is - tűnik a rendszer, ám tapasztalatok szerint a vírusirtók zseniálisak.

Ha előbukkan néhány biztonsági luk, a felhasználók bizalma hamar megcsappanhat, elvégre a hálózaton letöltött programok hiányos védelmi rendszer mellett potenciálisan hatalmas zűröket okozhatnak a helyi rendszerben.

Mindezen kérdések ellenére bizalommal lehet tekinteni a Jáva jövőjére. [8]

3.2 Néhány főbb specialitás a javaban

3.2.1 A program szerkezete

A programok *import* utasításokkal kezdődhetnek, felsorolva a programban felhasznált könyvtárak nevét. Ez nem kötelező, az egyes könyvtári elemekre való hivatkozásnál is megadhatjuk a könyvtár nevét.

C programozók figyelem: ez nem *#include* parancs, nem a fordítóprogramnak szóló üzenet a forrásszöveg beolvasására. Az *import* a kapcsolatszerkesztőnek, betöltőnek szól, már lefordított kódú könyvtárakra hivatkozik. Importálásnál csomagokból (*package*) osztályokat (*class*) importálunk, azaz a programunkban felhasználhatóvá tesszük. A csillag az adott csomag összes osztályát jelenti.

```
import java.net.*;  
import java.io.*;  
import java.lang.*;  
import java.util.*;
```

Ezután rögtön egy osztálydeklarációnak kell következni, a Jáva programban minden változó, minden utasítás csak osztályok törzsében szerepelhet.

3.2.2 Változók és értékek

Az osztályok belsejében változókat deklarálhatunk, a deklaráció szintaxisa egy - két módosító alapszó (*static* vagy *final*) kivételével megfelel a C - nek:

típus változónév [= kezdeti_érték]

A nyelv tartalmaz néhány beépített, egyszerű adattípust, amelynek neve és értékkészlete a következő táblázatban található:

egész típusok	byte	8 bit kettes komplement
	short	16 bit kettes komplement
	int	32 bit kettes komplement
	long	64 bit kettes komplement
valós típusok	float	32 bit IEEE 754 lebegőpontos
	double	64 bit IEEE 754 lebegőpontos
karakter típus	char	16 bites Unicode karakter kód
logikai	boolean	<i>true</i> vagy <i>false</i>

Látható, hogy csak előjeles egész típusok vannak, bár ez nem jelenti azt, hogy nem lehet rájuk például a szokásos bitműveleteket alkalmazni. A C - től eltérés az is, hogy itt az egyes beépített típusok mérete, helyfoglalása nyelvi szinten definiált, nem függ a programot futtató gép architektúrájától. A karakterek tárolásánál a nemzetközileg elfogadott, 16 bites *Unicode* - ot használják.

```
static final int SMTPport = 25;
```

A Jávában nincs *#define*, ennek a legközelebbi megfelelője a *final* változó, a fordítóprogram gondoskodik róla, hogy ennek értéke ne változhasson meg. Látható,

Szállítványozó és raktározó céget irányító és támogató program

hogy a változó deklarációjával együtt kezdeti értéket is adhatunk neki, ez persze *final* változó esetén kötelező.

A *static* módosítók azonosak a C++ -ból ismert osztályváltozókkal és ezek közelítik meg leginkább a szokásos nyelvek globális változóit.

A beépített numerikus típusokon a C -ből jól ismert műveletek értelmezettek. A numerikus típusú értékekkel a szokásos aritmetikai műveletek használhatók. Az egész típusú értékekre használható bit műveletek kiegészültek a `>>>` operátorral, amely jobbra léptetésnél 0 -t és nem az előjelet lépteti be a legnagyobb helyiértékű bitre. Ezek az operátorok értékadással is kombinálhatók, pl.: `+=`, `/=`, ...

A logikai értékek a C -vel ellentétben nem egész típusúak. Azonos típusú értékek összehasonlítására használt operátorok logikai értéket állítanak elő, ilyen értékekre használhatók az ismert logikai műveletek is.

A következő táblázat tartalmazza a Jáva nyelv összes operátorát:

<code>() [] .</code>	zárójelek, indexelés, osztály komponensére hivatkozás
<code>++ --</code> <code>! ~</code> <code>instanceof</code>	(pre- vagy poszt-) inkremens és dekremens, logikai és bitenkénti negálás, típus ellenőrzés
<code>new (type)expression</code>	új egyed létrehozása
<code>* / %</code>	szorzás, osztás, moduló
<code>+ -</code>	összeadás, kivonás
<code><< >> >>></code>	bitenkénti léptetések
<code>< > >= <=</code>	összehasonlítások
<code>== !=</code>	egyenlő, nem egyenlő
<code>&</code>	bitenkénti AND
<code>^</code>	bitenkénti XOR
<code> </code>	bitenkénti OR
<code>&&</code>	logikai AND
<code> </code>	logikai OR
<code>? :</code>	feltételes kifejezés

<code>= += -= *= /= %= ^= &= = <=> >>= >>>=</code>	Különböző értékadások
--	-----------------------

Az egyszerű, beépített típusokon túl a Jáva csak kétfajta összetett adattípust tartalmaz, a programozók által definiálható osztályokat (*class*) illetve a tömböket (*array*). Nincs *struct*, *union*, de nincs *typedef* sem, sőt mutatók (*pointer*) sincsenek!

```
static Socket smtpconn;  
static DataInputStream instream;  
static DataOutputStream outstream;
```

A fenti 3 sor a szabványos Jáva könyvtárakban definiált osztályok - *Socket*, *DataInputStream*, *DataOutputStream* – egy - egy példányának, egyedének (objektum) foglal helyet, pontosabban ezek egyelőre csak üres hivatkozások, az objektumok még nem jöttek létre.

3.2.3 A főprogram

Minden Jáva alkalmazásnak, pontosabban az egyik osztályának - mivel a Jávában minden utasítás csak osztályok definíciójában szerepelhet - tartalmaznia kell egy *main* nevű, itt következő fejlécű függvényt:

```
public static void main (String args[])
```

A *public* a módszer láthatóságáról nyilatkozik, jelentése: bárhonnan meghívható. A *static* eljárás csak osztályváltozót használ, a *void* a módszer visszatérési értékének típusa: nem ad vissza értéket.

A Jáva alkalmazás futtatását a virtuális gép a *main* eljárás végrehajtásával kezdi. A *main* eljárás paramétere egy szövegtömb, amelynek egyes elemei a parancssorban megadott argumentumokat tartalmazzák. A szövegek tárolására, kezelésére szolgáló *String* egy előre definiált osztály. A tömbök indexértékei nullától kezdődnek, viszont itt az *args[0]* az első argumentumot adja vissza. A Jáva tömbök méretét futás közben is lehet kérdezni.

Szállítványozó és raktározó céget irányító és támogató program

Az egyes eljárásokban természetesen használhatunk lokális változókat, itt a *String* osztály egyedére lesz szükségünk, amelyet deklarációjával egyidejűleg a *new* parancs segítségével létre is hozzuk.

```
String res = new String();
```

Lokális változókat a módszerek törzsében tetszőleges helyen deklarálhatunk, sőt az egyes programblokkok - *{ ... }* - saját, máshonnan nem látható lokális változókkal rendelkezhetnek.

3.2.4 Vezérlési szerkezetek

Meg kell vizsgálnunk, hogy a programunkat megfelelő számú argumentummal hívták - e meg! Amennyiben a programnak nem 2 argumentuma volt - az SMTP kiszolgáló és a keresett felhasználó neve -, tájékoztató üzenetet írunk ki és befejezzük a program működését.

```
if (args.length != 2)
{
    System.out.println("Usage:");
    System.out.println(" java SMTPclient <host> <user>");
    System.exit(1);
}
```

A *System* könyvtár a Jáva programok futtatásához szükséges objektumokat, módszereket tartalmazza. Ilyen a programok szabványos kimenetét (*standard output*) megvalósító *out* objektum, a kiíró eljárás, a *print*, illetve *println*.

A programból vagy a *main* módszer befejezésével, vagy a *System* könyvtár *exit* módszerének meghívásával lehet kilépni.

A Jáva nyelv átvette a C vezérlési szerkezeit, az elágazásokhoz az *if - else* és *switch*; ismétlésekre, ciklusszervezésre a *for*, *while* és *do-while* utasítások használhatók.

3.2.5 Fordítás, futtatás

A programot pl.: *SMTPclient.java* néven, lefordítani a

```
javac SMTPClient.java
```

paranccsal lehet. Ha nem követtünk el gépelési hibát és a Jáva Fejlesztői Környezetet (*JDK*) is helyesen telepítettük, akkor a fordító létrehoz egy *SMTPClient.class* állományt, ami a lefordított Byte kódot tartalmazza. Az állomány neve nem a forrás nevéből származik, hanem az állományban definiált osztály nevét hordozza. Ha több osztályt definiálnánk, a fordító több különálló állományt hozna létre.

3.2.6 Absztrakt adattípusok létrehozása

A Jáva nyelvben csaknem minden objektum. A nyelv ugyan tartalmaz néhány egyszerű adattípust, de ezeken felül minden egyéb adatszerkezet, vagy valamilyen osztályba tartozó objektum, vagy tömb, amely speciális osztály. Mellesleg a nyelv tartalmaz a beépített típusokat becsomagoló osztályokat (*wrapper class*) is: *Boolean*, *Character*, *Double*, *Float*, *Integer*.

3.2.7 Objektumok

Az objektumok olyan programösszetevők, amelyek szoros egységbe foglalják az állapotukat leíró belső adatszerkezetüket és a rajtuk értelmezhető műveleteket. Ebben az értelemben az egyes objektumok nagyon hasonlítanak egy adott típusba tartozó értékekhez. Egy *int* típusú érték is elrejtje előlünk a belső reprezentációját és csak az előre definiált műveletek végezhetők rajta.

A Jávában minden egyes objektumnak meghatározott típusa kell, hogy legyen, azaz valamelyik osztályba kell tartoznia. Az objektumokra változókon keresztül hivatkozhatunk, pl. a

```
Person somebody;
```

azt jelenti, hogy a *somebody* név mindig egy *Person* típusú objektumra hivatkozik. A fenti deklaráció nem jelenti egy új objektum létrejöttét, a változó egyelőre nem hivatkozik semmire, azaz "értéke" *null*. A

```
Person somebody = new Person("Kovacs", "Istvan", "15502280234");
```


utasítás nem csak definiál egy új változót, de a *new* paranccsal létre is hoz egy új objektumot, amelyre a változó hivatkozik.

A Jávában minden változó hivatkozást tartalmaz csupán. Ezt figyelembe véve érthető, hogy az értékadás (=) operátor nem készít másolatot, a bal oldali változó ugyanarra az objektumra hivatkozik majd, mint a jobb oldal. Ha valakinek mégis másolatra van szüksége, többnyire használhatja a csaknem minden osztályban megtalálható *clone* módszert. Hasonlóképpen az egyenlőség vizsgálatának operátora (==) is a referenciák egyenlőségét ellenőrzi, az objektumok strukturális ellenőrzésére az *equal* módszer való.

Azt már láttuk, hogy új objektumokat a *new* utasítással lehet létrehozni, megszüntetni, törölni viszont nem kell. A Jáva virtuális gép, amely a programokat futtatja, tartalmaz egy szemétgyűjtő (*garbage collection*, *gc*) - jelenleg az ún. *mark-and-sweep* típusú - algoritmust, amely általában a háttérben, a CPU szabadidejében, a programmal aszinkron futva összegyűjti a már nem hivatkozott objektumokat, és azok tárterületét felszabadítja, hogy a *new* újra felhasználhassa majd. A szemétgyűjtés teljesen rejtetten, automatikusan történik.

Az egyes objektumok módszereit az

```
obj.method(parameter, parameter, ...)
```

szintaxissal hívhatjuk meg. Az objektumorientált terminológia ezt üzenetküldésnek (*message passing*) nevezi. Az ilyen módszerhívások vezethetnek az objektum belső állapotának, azaz belső változói értékének megváltozásához is, de gyakran csak egy belső változó értékének lekérdezése a cél.

Bár a tiszta objektumorientált elvek szerint egy objektum belső állapotváltozóinak értékéhez csak módszerein keresztül lehetne hozzáférni, a Jáva megengedi, hogy bizonyos változókat közvetlenül is elérhetővé tehessünk.

3.2.8 Osztályok definíciója

A Jáva programozók osztályokat a

```
class ClassName { /* az osztály törzse */ }
```

programszerkezettel definiálhatnak, ahol a *ClassName* az újonnan definiált osztály neve lesz, az objektumok belső állapotát és viselkedését pedig a kapcsos zárójelek közötti programrészlet írja le. A *class* alapszó előtt, illetve az osztály neve után opcionálisan állhatnak még különböző módosítók.

3.2.9 Egyedváltozók

Az osztály törzsében szerepelnek azok a változó - deklarációk, amelyek az egyes egyedek belső adatszerkezetét valósítják meg. Ezeket gyakran egyedváltozóknak (*instance variable*, *member variable*) nevezzük, jelezve, hogy az osztály minden egyede ezekből a változókból saját készlettel rendelkezik. A változó-deklarációk:

```
class Person
{
    String  name, surname, pid;
    byte   age;
    Person  mother, father;
```

A képzeletbeli *Person* típusunkba tartozó egyedek három szövegváltozóban tárolhatják a személy vezeté- és keresztnévét, személyi számát, egy Byte - ban az életkorát és két *Person* típusú változóban az apját és anyját. Az utolsó 2 változó példa arra, hogy egy egyed belső állapota hivatkozhat más - akár az egyeddel megegyező típusú - objektumokra is.

3.2.10 Osztályok leszármaztatása

Ha egy új osztályt már valamelyik meglévő alapján akarjuk definiálni, azt az osztály fejében a következő szerkezettel kell jelezni:

```
class Child extends Parent { /* Child törzse */ }
```

Az ily módon megadott, ún. leszármazott osztály (*Child*) örökli a szülője (*Parent*) tulajdonságait, azaz a belső állapotát leíró adatszerkezetét és a viselkedést megvalósító

módszereit. Természetesen az osztály törzsében ezt tovább bővítheti, a viselkedését módosíthatja, ezt sugallja az *extends* (kiterjeszt) kifejezés a definíciónál.

A Jáva az ún. egyszeres öröklődést engedélyezi, azaz minden osztálynak egyetlen közvetlen szülője lehet. Tulajdonképpen nem csak lehet, de mindig van is szülője, a nyelv ugyanis definiál egy beépített *Object* nevű osztályt, amennyiben az új osztály definíciójában máshogy nem rendelkezünk, automatikusan "*extends Object*" értendő, azaz minden osztály közvetlenül vagy közvetve, de öröklí az *Object* - ben definiáltakat. Az így örökölt módszerek például az egyes objektumok másolásával (*clone*), összehasonlításával (*equals*), megnevezésével (*toString*), illetve osztálya megkülönböztetésével (*getClass*), vagy a többszálú futtatással (pl. *wait*, *notify*) kapcsolatosak.

Szükség esetén a meglévő osztályainkból való leszármaztatást megtilthatjuk úgy, hogy azt "*final class*" - ként definiáljuk. Ezt akkor alkalmazzuk, ha nem akarjuk, hogy egy mások által, belőle leszármaztatott osztály nem kívánt módon módosítson az osztályunkban definiált viselkedést. [9]

3.3 A grafikus interfész

Manapság szinte minden program eseményvezérelt, és grafikus a felhasználói interfésze. A grafikus felhasználó felület (Graphical User Interface, GUI) programozására a Java kétféle osztálygyűjteményt (keretrendszert) ad, az AWT – t és a Swing - et. Az AWT a régebbi rendszer, azt már nem fejlesztik tovább. A Swing az AWT - re épülő újabb osztálygyűjtemény. A Java tervezői mindkettőben arra törekedtek, hogy a felhasználói interfész platformfüggetlen legyen. A két osztálygyűjtemény jellemzői:

- AWT (Abstract Window Toolkit, absztrakt ablakozó eszköztár): Ez az osztálygyűjtemény felhasználja az operációs rendszerek saját, natív (gépi kódú) GUI komponenseit. A natív (másképp: peer) komponensek meghívásával a Java az éppen használt operációs rendszerre jellemző küllemet ad a grafikus felhasználói felületnek. A platformfüggetlenséget az adja, hogy a rendszerhívások révén mindig a pillanatnyi platform ún. nehéz. súlyú (heavyweight) komponensei futnak. Az AWT - osztályok a java.awt csomagban találhatók.

- Swing: A Java fejlesztői időközben rájöttek, hogy a peer komponensek hívása mégsem ideális, mivel a különböző platformokon más és más képernyőelemek jelentek meg. Ezért megírták az AWT - komponensek pehelysúlyú (lightweight, swing) változatát, és ki is bővítették a gyűjteményt. A pehelysúlyú komponensek már nem használják az operációs rendszer natív elemeit - a Swing grafikusan rajzolja meg a komponenseket. A Swing az AWT - re épül, s intenzíven használja annak nem látható elemeit. A kirajzolás miatt a Swing - komponensek megjelenése sokkal lassúbb, mint a natívaké; a hardverek fejlődésével azonban ez nem sokáig lesz majd nehézség. A képernyőn megjelenő Swing - komponensosztályok azonosítója J betűvel kezdődik - például az AWT - ben deklarált TextField - nek a JTextField a swingbeli párja. A Swing - osztályok a javax.swing csomagban kaptak helyet.

A Swing alkotói a pehelysúlyú komponensek megalkotásában felhasználták a már meglevő AWT - komponenseket; a pehelysúlyú komponensek pókhálószerűen vannak rátelepítve ezekre a komponensekre. A Swing osztályhierarchiáját nem sikerült olyan kristálytisztán kialakítani, mint az AWT - ét. Az AWT azonban már elavult, taníthatóságán kívül nem sok előnye maradt (bár az appletekben még használják). A Swingbe rengeteg csábító dolog van beépítve, a programozónak könnyű rájuk szoknia. Ilyen többek között az ikonok használata a címkéken és a nyomógombokon, valamint a grafikus elemeket gyorsabban megjelenítő kettős pufferelési technika. Nem utolsó szempont a szabványos dialógusablakok megjelenése sem. A felhasználói interfész komponensei a képernyőn látható objektumok: a keret, a nyomógomb vagy a beviteli mező. A komponensek programozásához láthatatlan, nem komponens osztályokat is felhasználunk, olyanokat, mint a pont, a téglalap, a szín és a betűosztályok.

3.3.1 Komponensek és tulajdonosi hierarchiájuk

Egy Java alkalmazás grafikus felhasználói interfésze (Graphical User Interface, GUI) komponensekből áll. A komponensek téglalap alakú képernyőfelületek, amelyeknek meghatározott tulajdonságaik vannak (elhelyezés, méret, szín, láthatóság stb.), és előre megadott szabályok szerint reagálnak különböző eseményekre (egér-,

billentyűesemény stb.). A komponens osztálya mind a java.awt.Component leszármazottja. A Swing – komponenseknek közös ősük a javax.swing.JComponent. Vannak olyan komponensek, amelyekre újabb komponensek tehetők, ezeket konténerkomponenseknek nevezzük - ilyen például az alkalmazás kerete és a dialógusablak. A vezérlőkomponensek nem bonthatók tovább - például a nyomógomb és a szövegmező. A felhasználói interfész komponenseinek jól meghatározott tulajdonosi (tartalmazási) hierarchiájuk van. [10]

4 Az elkészült Program ismertetése

4.1 Amiket a grafikus interfészből használtam

Ezen ismertető csak a legérdekesebb eseteket ismerteti a program komplexsége miatt.

A legfejlettebb technika amit a grafikus interfészből használtam az a több szálon való futása a programnak. A legtöbb számolást végző modulom indíthat el egyszerre több számolást is, íme a kódrészlet:

```
if(!bool && !isOpen)
{
    TSPopti=new Thread(newTSPopt(utakTomb,varossz,""));
    TSPopti.setPriority(1);
    TSPopti.start();
}
```

A feltételek teljesülése estén, elindulhat az adatokkal való számolás, amit a TSPopt eljárás végez. Létrehoztam egy új szálat, a prioritását a leggyengébbre állítottam, hogy minél kevésbé akadályozza a többi modulban való munkát, mivel azok polinomiális idejűek, így gyorsabbak. Azokhoz is adatbevitelre van szükség, a főlétsleges időben a processzor a TSPopt kiszámolásával törődhet.

Két fő eseményfigyelő komponensre volt szükségem a programhoz. Mouse figyelőre, és fókusz esemény figyelőre. Billentyűzet esemény figyelőre nem volt szükségem, így a fő frame osztályom feje következőképpen néz ki:

Szállítmányozó és raktározó céget irányító és támogató program

```
class HaromPanel extends JPanel implements ActionListener,  
FocusListener  
{
```

Az adatok bevitelére mind a három modulnál JTextField – eket használtam, amelyek beviteli hosszát egy speciális limitáló class – al határoztam meg.

```
class JTextFieldLimit extends PlainDocument  
{  
    private int limit;  
    private boolean toUppercase = false;  
    JTextFieldLimit(int limit)  
    {  
        super();  
        this.limit = limit;  
    }  
    JTextFieldLimit(int limit, boolean upper)  
    {  
        super();  
        this.limit = limit;  
        toUppercase = upper;  
    }  
  
    public void insertString  
    (int offset, String strcsucsok, AttributeSet attr)throws  
    BadLocationException  
    {  
        if(strcsucsok == null) return;  
        if((getLength() + strcsucsok.length()) <= limit)  
        {  
            if(toUppercase) strcsucsok = strcsucsok.toUpperCase();  
            super.insertString(offset, strcsucsok, attr);  
        }  
    }  
}
```

```
        }  
    }  
} //class JTextFieldLimit
```

A három modul mindegyike a fő frame -ből egyenlő szeletet birtokol hosszanti irányban. Az elrendezésük többféle lehet, ezek közül a `FlowLayout` és a `GridLayout` –ot használtam, vagyis a folytonos és a rácsos elrendezést. Rácsos elrendezésű például a három modul megjelenése, folytonos elrendezésűek pedig a modulok komponensei.

Használtam még illusztráló képeket, és gif animációkat a design kedvéért.

4.2 TSP modulom

4.2.1 Bevezető

Íme először röviden a stratégiám, majd a részletesebb ismertetés, utána a kódot összehasonlítom egy már meglevő algoritmussal, amit más már megoldott:

Tegyük fel, hogy adott a TSP probléma egy példánya n várossal és d_{ij} távolságokkal. A városok minden olyan S részhalmazára, amely nem tartalmazza 1 -et, és minden j eleme S városra, legyen $c[S,j]$ annak az 1 -ből j -be vezető legrövidebb útnak a hossza, amely S minden elemét egyszer érinti.

Megadtam egy olyan algoritmust, mely a dinamikus programozás módszerével kiválasztja a $c[S,j]$ értékeket a kisebbtől a nagyobb S halmaz felé haladva. [11]

A kód minden lépésére, vagyis egy újabb útvonal ellenőrzéshez a program (TSPopt) több függvényen illetve eljáráson megy keresztül. A hívási láncolat legtetején a következő, egy csúcssal hosszabb út létrehozása áll, majd az így kapott számokból (csúcsok) álló sztringet hozzáadom egy láncolt listához. A programom egyszerre egyenlő hosszúságú utakat tárol. Amint hozzáadtuk a következő utat a láncolt listához egy újabb függvény segítségével megkeressük, hogy a listában van e már olyan út, amely ugyanazokat a csúcspontokat tartalmazza. Ha talál ilyet, akkor megnézi, hogy az aktuális sorrend, vagy ez, a megtalált tartalmazza a rövidebb menettávot. És eszerint

Szállítmányozó és raktározó céget irányító és támogató program

járok el tovább. Ha az új rövidebb, akkor azt tapasztom a listához és a régit törlöm, ha hosszabb nem teszek semmit, vagyis marad a rövidebb változat a listában.

Íme egy eljárás amelyhez hosszabb magyarázatot adok:

```
public void dinkiscsucspast(String strcsucok,String strcsucokord)
{
    list1.addLast(strcsucokord);
    list1.addLast(new Integer(partuthossz));
    list1.addLast(strcsucok);
}
```

Az eljárás paraméterei az aktuálisan vizsgált út, és ezen út csúcsaik növekvő sorrendbe rendezve. Azért kell a csúcsokat növekvő sorrendbe is a listához csatolnom, hogy kiderüljön az utak közti tartalmazási hasonlóság, egy ellenőrzéssel, ahogy azt fentebb említettem.

A úthosszat a másodikként csatolt globális, Integer objektum tárolja, végül magát az utat harmadikként csatoljuk a listához.

Lássunk egy inputot és eredményét a program által:

Bemeneti város szám: 5 (Ezután egy entert nyomunk)

1->2:7 1->3:6 1->4:45 1->5: 78 2->3:65 2->4:45 2->5: 76 3->4:8 3->5:8 4->5:7

Ekkor, ha jól adtuk meg az adatokat, akkor egy ablakban kapjuk az eredményeket: Az egyik legrövidebb körutat és hosszát, grafikusan is szemléltetve egy sokszöggel:

Út: 1 -> 2 -> 4 -> 5 -> 3 -> 1

Hossz: $7+45+7+8+6 = 73\text{km}$

4.2.2 Összehasonlítás más algoritmusokkal

Valószínűleg hatékonyabb eljárást kapnánk, ha a láncolt listához egyszerre egy objektumot csatolnánk, mely tartalmazná a három fent említett, listához adott objektumokat. Az indexelés is gyorsabban menne ez által.

Egy másik nagy különbség pedig a visszakeresés körül forog. Valószínűleg gyorsabb lenne a program futása és elérnénk lefele a $n^2 2^n$ futási időt, ha az utakat egy n fiút tartalmazó kereső fával állítanánk fel. Így sokkal gyorsabb lenne a hasonló út megtalálása, hiszen csak n^2 lenne a keresési idő, mert csak végig kell menni a tömbön, listán és így tovább lefelé a fán, egészen, míg meg nem találjuk a keresett hasonló utat.

Ha a programomba beépítve lenne ez a technika, akkor k – adik részút, vagyis egy szintnek a generálása továbbra is (n alatt a k) lépést igényelne, ugyanis ennyi féle képpen lehet az n városból kiválasztani k várost tartalmazó részúthosszat. Ha csak tehát az algoritmusom ezen részének összes lépésszámát vizsgálom, akkor ez – tehát az

összes szintet kigenerálva: 2^n , hiszen $\sum_{k=0}^n (n \text{ alatt a } k) = 2^n$. Ehhez, ha hozzávenném a

fent említett új technikát, ha legrosszabb esetet vizsgáljuk, márpedig egy jó programozó mindig ezzel számol, akkor minden egyes újabb útszakasznál ellenőrzöm tehát, hogy volt – e már ez az útrészlet, ez a keresés n^2 , így akkor, felülről becsülve valóban $n^2 2^n$ lenne a futási idő.

4.2.3 Egy program bemutatása a TSP - re

Egy fent említett, Java program egy tetszőleges számú állomás között megvalósítható körutazási feladat közelítő megoldását adja meg Dacey - vagy Dacey – Vogel (fent) módszer alkalmazásával. A program futásához szükséges egy mátrix, amely az egyes csomópontok közötti költséget (mely lehet akár távolság vagy idő) adja meg.

A programhoz nem kötődnek szorosan mértékegységek, ezért tetszőleges költség alapú számításra használható. A program felülete 3 egységre tagolódik:

- Távolságmátrix:

Itt tudjuk a táblázat adatait feltölteni. A táblázat elemeiként kizárólag csak számok adhatóak meg (a program nem engedélyezi betűk felvitelét a mátrixba). Ugyanakkor nem feltétel, hogy a mátrix szimmetrikus legyen, hiszen nem minden

Szállítmányozó és raktározó céget irányító és támogató program

esetben ugyan azok az adatok (pl.: idő szempontjából nem mindegy, hogy milyen domborzati viszonyok vannak az oda -, illetve a visszaút során).

- Algoritmus:

Kiválaszthatjuk a Dacey- vagy a Dacey - Vogel algoritmust, attól függően, hogy melyik módszerrel szeretnénk meghatározni az útvonalat. Ezáltal lehetőség nyílik arra, hogy azonos bemeneti mátrix esetén összehasonlítsuk a két módszer által meghatározott eredményt.

- Útvonal: A megfelelő algoritmus kiválasztása után itt tekinthetjük meg a program által meghatározott állomások sorrendjét és a körút teljes hosszát.



1.ábra Egy TSP program

A program eszköztára: a táblázattal végezhető műveletek parancsikonjait tartalmazza. A program eszköztára a következő funkciókat biztosítja:

-  Új üres mátrix létrehozása: Egy olyan mátrix kerül létrehozásra, mely nem tartalmaz egyetlen helyszínt sem.
-  Adatok betöltése: Előzőleg mentett adatok felvitele a mátrixba.

Szállítmányozó és raktározó céget irányító és támogató program

-  Adatok mentése: Az éppen aktuális mátrix mentése.
-  Új hely hozzáadása: Új helyszínnel bővíthetjük a mátrixot. Az új elem nevét egy felbukkanó ablakban adhatjuk meg.
-  Hely törlése: Törli az aktuális helyet a mátrixból.
-  Hely átnevezése: A kiválasztott hely nevét változtathatjuk meg.
-  Hely tiltása: Ezzel a parancsikkal figyelmen kívül hagyhatunk bizonyos helyszíneket a számítások során.
-  Hely engedélyezése: Előzőleg tiltott helyszínt újra figyelembe vesz.
-  Mozgatás felfelé: A kiválasztott helyet egy pozícióval feljebb mozgatja (sora eggyel felfelé, oszlopa eggyel balra tolódik).
-  Mozgatás lefelé: A kiválasztott helyet egy pozícióval lejjebb mozgatja (sora eggyel lefelé, oszlopa eggyel jobbra tolódik).
-  Útvonal meghatározás: A kiválasztott algoritmus segítségével az adott költség mátrix alapján meghatároz egy lehetséges körútvonalat. [2.2]

4.2.4 Összehasonlítása a modulomnak a fenti programmal.

A fenti megoldás is lehet hatékony, legfőképp ha nincs más lehetőséggel, program modulokkal kiegészítve úgy, mint a programom, különben nehezen lehetne áttekinteni, mint egy egyszerűen használható programot. A programom egyébként, egészében összehasonlítható az ilyen változatokkal, hiszen itt dinamikusan bővíthető lenne a városok, ládák, stb. száma. Tényleg hatékony, hogy ha már megvan egy városhalmaz a távolságmátrixával, akkor, ha a mátrix valami oknál fogva bővül, akkor felhasználóbarát módjára, elég csak az új tételt megadni és a hozzátartozó távolságokat, ami hozzácsatlható az eredeti mátrixhoz. Nem látom viszont ebben a megoldásban az áttekinthetőséget, ha már megadtunk egy városhalmazt, akkor az esetleges hibásan megadott él súlyát, nehéz, vagy egyáltalán nem lehet módosítani. Vagyis egyszerre nem tekinthető át a távolságmátrix, ezáltal válik nehezzé a hibák módosítása. Egyszerre nem tekinthető át teljességében és nem is lehet megadni teljességében a távolság mátrixot, csak városonként. Az én programomban viszont igen, ezáltal elkerülhető, hogy hiba csússzon a bevitelbe. A programom egy képernyőből áll, a főmunkaterületből, ha megadtuk a városok számát, akkor azt már nem lehet bővíteni egy újabbal, viszont áttekinthető az egész, és szkrollozható a képernyő nagyobb bemenet esetén.

Szállítmányozó és raktározó céget irányító és támogató program

Annak nem sok értelmét látom, hogy a távolságmátrixban eggyel feljebb, vagy lejjebb lehet tolni a városokat, de annak sem, hogy azokat el lehet nevezni, - az egyszerűségből, tényszerűségből veszt ezáltal a program.

A programom hatékonysága a globalitáson nyugszik, egyszerre adhatóak meg az adatok, ezáltal áttekinthetőek, és ha kell, így egyenként, a többi adathoz láthatóan viszonyítva javíthatóak. Egyelőre szimmetrikus mátrix adható meg benne, feltételezve az oda – vissza vezető út egyenlő hosszát.

Persze nem látom valószínűtlennek a fenti program és az én programom keverését, vagyis a programom fejlesztetőségét.

4.3 Ládarakás

4.3.1 Specifikáció:

Egy konténer raktárban, egy sorban, egymás mellett tárolnak n darab konténert. A raktáros át akarja rendezni a konténereket úgy, hogy néhányat egymásra rak. Az átrendezést bizonyos konténerek egyesével történő átrakásával lehet végezni. Ha az átrendezés során az i - dik konténerhelyen lévő konténert a j - dik konténerhelyre rakja át, ennek költsége $s_{i|j}$, ha a konténer súlya s_i . A raktáros olyan átrendezést akar, amely után pontosan k konténerhelyen lesz konténer (esetleg egymásra rakva). A raktáros az optimális átrendezést keresi, tehát amelyre az összköltség minimális. Ez a modul kiszámítja a raktár optimális átrendezésének költségét, a továbbiakban ezt a tulajdonságot vizsgálom, de az optimális rakások helyeit is meghatározza!

4.3.2 Hatékonyság:

A probléma (n alatt a k) futási időn belül, a nyers erő módszerével megoldható, ugyanis ennyiszor lehet k darab, egymás mellett lévő ládából, halmazt kiválasztani úgy, hogy a halmazoknak van egy kitüntetett pontja, amelyre pakoljuk a halmaz többi ládáját. Tehát, vesszük a kitüntetett pontok (k) és az áthelyezendő ládák ($n - k$)

Szállítványozó és raktározó céget irányító és támogató program

kombinációját ami egy (n) k – ad osztályú kombináció. Tudva hogy a ládák sorrendje mindig kötött: A $0 - k$ $(n - k)$ az áthelyezendő ládák, az $x - ek$ (k) a rakások helyei:

$$n = 10, k = 3 - ra:$$

000x00xx00, x00x00000x, 0x000000xx, 0x0x0x0000 ...

$$\frac{n!}{k_1!k_2!} \text{ ahol } k_1+k_2=n, \text{ vagyis ebben az esetben } n \text{ alatt a } k.$$

Persze dinamikus programozással a megoldás már algoritmikusan megoldható lesz, felülről becsülve közel n^4 futási idejű, ami beláthatóan jobb futási idő elég nagy bemenetekre. Ez a dinamikus megoldást választottam én is lent.

Íme a részleteket nem kímélve a megoldási stratégiám:

4.3.3 Stratégiám

Minden i, j - re kiszámoltam a $T(i, j)$ értéket, ami a minimális rakodási költség azon feltételek mellett, hogy legfeljebb i különböző helyre rakodunk, és az utolsó rakodási hely a j - dik konténerhely. A végső válasz, akkor a $\min\{T(k, i) : i=1, \dots, n\}$, ahol n a konténerek száma, k pedig a rakodásra megengedett helyek száma. Ezt a $T(i, j)$ - t kell dinamikus programozással, táblázat kitöltéssel végigszámolni.

A döntés, ami alapján a rekurziót felírható, a j - dik konténerhely előtt melyiket használtuk. Tehát minden $r < j$ - re ki kell számolni mi a költség, ha r - t használjuk j előtt. Ezt a költséget úgy kaphatjuk meg, hogy venni kell a $T(i-1, r)$ értéket (ez azt adja meg, mennyi a rakodási költség, ha $i - 1$ helyet használva az r - dik az utolsó. Meg kell határozni kiket rakunk az új megoldásba az r - hez, kiket rakunk a j - hez (az $r+1, r+2, \dots, r+p/2$) helyek kerülnek r - hez ahol $p = j - r$, a többiek az új megoldásban j - hez

lesznek rendelve r helyett. Azt az r - et választjuk, amire a költség minimális és a hozzá tartozó költséget.

Íme egy input:

Megadjuk először a rakások számát: 4

Meg kell adni a maximum 15 helyen lévő ládák súlyát, ami 0 is lehet:

Pl.:

1:70 2:66 3:0 4:0 5:6 6:65 7:5 8:54 9:7 10:5 11:3 12:0 13:45 14:65 15:0

Az entert megnyomva grafikusán támogatva kétféle információt is kapunk:

A rakások helyei: 1 6 8 14

És a minimális munkaköltség: 148e.

4.4 Ládapakolás

4.4.1 NP – nehéz megoldások

Először ismertetem a saját, tehát a hagyományos off – line problémára az egyik exponenciális futási idejű algoritmust. A cél az tehát, hogy minél kevesebb 1 egységnyi konténert használjunk fel. Az algoritmus először megvizsgálja, hogy 1 ládába bele fér – e az összes tárgy, ha nem, megvizsgálja, hogy kettőbe belefér – e, ha nem, háromba stb. Ezt úgy teszi, hogyha n a tárgyak száma és k db konténerrel tartunk, veszi a összes tárgy összes lehetséges k számú halmazát, a halmazokban tetszőleges számú tárgy lehet, de legalább egy. Ez, mint tudjuk $(n$ alatt a $k) * (k)^{(n-k)}$ db számú lenne, ha érdekes lenne a tárgyak bepakolási sorrendje. De kihasználva, hogy a bepakolási sorrend mindegy a ládákban, egy jó algoritmussal, - mely részleteit e kereteken belül nem ismertetem - elérhető lehet egy kisebb fokú NP – nehéz algoritmus. Hiszen egy n tárgyból álló rakásból mindössze 2^n db különböző halmazt lehet alkotni. A konténerbe kerülésük sorrendje nem változtat az eredményen, vagyis nem kell megvizsgálni a halmazokban lévő tárgyak minden permutációját, nem úgy, mint ahogy a TSP programomban azt meg kellett tennem, hiszen nem mindegy, milyen sorrendbe járom be, ott a már meglátogatott városokat.

Megállási feltétel: amint lesz olyan fent említett k db konténer, amelynél a halmazokban lévő tárgyak összmérete legfeljebb egy, abban a pillanatban megállítom

az algoritmust, és a problémára a válasz az aktuális k együttható lesz, és nem kell több. Vagyis minimálisan k db láda kell a tárgyakhoz.

Ha a fentebb említett vektorfedés változatra akarom megadni az algoritmust, - ahol az a fontos, hogy minél több legyen a legalább 1 összméretű halmazaink: csak úgy kell változtatni a fent említett algoritmust, hogy a leállási feltételt átírom, hogy ne akkor álljon meg a rekurzió, ha van olyan halmazsorozat ahol a halmazok összmérete ≤ 1 hanem, ha már nincs egy olyan halmazsorozat sem a szinten, amelynél a halmazok összmérete ≥ 1 . Ekkor a válasz a $k - 1$ együttható, vagyis maximálisan $k - 1$ db láda fedhető le.

4.4.2 Stratégiám, összehasonlítás

First - fit heurisztikát alkalmaztam a programomban. Ez az off - line probléma tehát, amikor is megvan már kezdetben a berakandó tárgyakról az információ. Hiszen a méretek adottak. Ez egy hagyományos probléma. A First - fit közelítő algoritmus sorba vesz minden tárgyat, és belerakja az első olyan ládába, amelyikbe befér. Az on - line algoritmusok esetében nem ismert az összes adat.

Az én ládapakolás modulom, a ládapakolás egy alap fajtája, asszimptotikus versenyképességi hányadosa 2, a fentebb említettek miatt.

Íme egy input:

1:6 2:30 3:76 4: 43 5:2 6:34 7:2 8:3 9:23 10:0 11:0 12:0 13:0 14:0 ...

Output: a tárgyak nagyságai grafikusán szemléltetve, és az eredmény, hogy hány láda kell.

A megoldás egyszerű: vesszük a tárgyakat és beletesszük az első olyan ládába, amibe még befér. A fenti tárgyak 3 ilyen ládában férnének el.

5 A programom hatékonysága

A mellékelt program grafikus felületen dolgozik. Amellett, hogy így tetszetősebb kivitelű, könnyebb a kezelhetősége, számos előnye van. Például a bevitt adatok javíthatósága. Konzolos felületen, vagyis nem grafikusán, körülményes lenne megoldani egy már bevitt adat javítását. Ezzel szemben grafikusán, a megfelelő beviteli mezőre kell csak fókuszálni akár mouse - al, akár tab - bal, shift - tab – al, és már javítható is. Tehát a program összeszedett, eleget tesz az emberközeliségnek, tesztelhetőségnek.

Gyors is, mert a mesterek technikáit alkalmaztam: heurisztikát, dinamikus programozást. Az alkalmazás egy szállítványozó program gyakran felmerülő problémáinak megoldására írtam. Rendkívül jól alkalmazható. Elindításakor, három részre osztott képernyő jelenik meg, amely a futás végéig megmarad, csak háttérszínét változtatják panelenként, eseménykor. A programba eseményfigyelők sora van építve, mint az egy grafikus felülettel rendelkező programhoz illik. Ezek figyelik például, hogy helyes számot adtam-e meg, nem ütöttem-e félre véletlenül gombokat, tehát a program eseményvezérelt, hatékony.

Interaktív is, kezdve onnan, hogy ismerteti az egyes panelek kezelését, működését, célját, kisebb változtatás után is megtekinthető az új eredmény, jelzi a hibás adatbevitelt, amely azonnal javítható, megmondja a hiba okát. Felhasználja a felbontás elvét is: kódban több önállóan használható osztály is van.

A program karbantartható is, hiszen tartalmaz final konstansokat, ellenőrző metódusokat, amelyeket átírva áttekinthetően befolyásolhatjuk a program paramétereit.

Az mondható el, hogy helyes is, mivel a három modul (panel) közül 2 is bizonyítottan egy optimális eredményt szolgáltat. Erre számos tesztet kipróbálása és hideg tesztelés a biztosíték.

A programot file műveletekkel is elláttam, ahogy azt a feladat kiírásban is szerepeltettem. Ez a panelek tetején lévő gombokkal kezelhető, úgy, mint mentés, betöltés. Egy általunk elnevezett állományba teszi az összes általunk megadott adatot, paramétert. Ezáltal hatékonyságot, kezelhetőséget értem el. Nagyobb számú adatok megadásakor, mikor azt csak egy - két helyen akarjuk módosítani, betöltjük a file betöltő párbeszéd ablak segítségével azokat és módosítunk. És ilyenkor is működik az újraszámolás, vagyis az eredmény bármilyen kisebb változtatás után is megtekinthető.

Nagy segítség a mentés akkor is, mikor például, már elszállították a termékeket és ezáltal, már nincs információnk a tárgyak méretéről, elég csak betölteni a ládapakolás modulba a megfelelő file – t.

A TSP – nél is ugyan ez a helyzet, ha folyamatosan mérjük és állapítjuk meg a városok közt a távolságokat, és ha ez esetleg napokat vesz igénybe, elég a már lement adatokat megadni, és már menthetők is. Később, mikor bővítjük a gráfot, vagyis további adatok birtokába kerülünk, az egyesek helyére már konkrét adatokat adhatunk meg.

6 Kitekintés

A programban sikerült feltérképeznem, ha nagy vonalakban is egy szállítványozó cég temérdek sok logisztikai feladataira szánt segítség néhány részét. A berakodástól a szállítványozásig. Meg kell említeni a mai világ egy új örületét, a genetikus algoritmusokat, amelyeket a TSP algoritmusoknál is egyre inkább próbálnak alkalmazni. A technikát nem rég óta próbálják alkalmazni, a természet, mint legokosabb alkotó, megoldó erő képében. A lényege, hogy megfigyelik például a fák törzsének rost növekedését, és például, eszerint építik meg, számítógép segítségével a hidak pilléreit. Vagy a repülő gépek tervezésénél a madarak szárny és farok felépítését veszik alapul, mikor az aerodinamikát akarják kiépíteni. Mindenképp én is ezt látom a jövő technikájának.

Irodalomjegyzék:

[1] <http://hu.wikipedia.org/wiki/Utazóügynök-probléma> 2010-03-04

[2.1,2.2]

http://www.szolnok.mtesz.hu/sztk/kulonszamok/2007/cikkek_pdf/Olah_Bela.pdf

2010-03-04

[3] <http://www.inf.u-szeged.hu/~cimreh/altpak.pdf> 2010-03-04

[4] <http://www.inf.u-szeged.hu/~cimreh/ladapak.pdf> 2010-03-04

[5] Thomas H. Corman, Charles E. Leiserson, Ronald L. Rives, Clifford Stien: Új Algoritmusok 15. fejezet bevezetője

[6] home.cab.u-szeged.hu -> pub/alg/I/n9.pdf 9.3 fejezet eleje 2006-07-04

[7]

http://www.diakoldal.hu/notes/36a5a6f51681318944206ead8ddf3284mesterseges_intelligencia.doc 2010-03-04

[8] <http://www.cab.u-szeged.hu/WWW/java/kiss/article.html#tc> 2010-03-04

[9] <http://www.info->

[bsc.netkey.hu/IR2254j/a_java_programozasi_nyelv_rejtelvei.doc](http://www.info-bsc.netkey.hu/IR2254j/a_java_programozasi_nyelv_rejtelvei.doc) 2010-03-04

[10] Angster Erzsébet Objektumorientált tervezés és programozás 2. II. fejezet , 5. alfejezet 105 – 106. oldal

[11] Papadimitriou C. H. Számítási bonyolultság 19. oldal 1.4.15. Feladat
2010-03-04

Ábrák: a fent jelzettekben belül

Nyilatkozat

Alulírott informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Tanszékcsoport, szoftverfejlesztés Tanszékén készítettem, programozó matematikus diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem könyvtárában, a kölcsönözhető könyvek között helyezik el.

2010-05-06

Dinnyés Balázs Emil

Köszönetnyilvánítás

Minden tiszteletemet és köszönetemet a témavezetőm Dr. Schrettner Lajos Egyetemi Adjunktusnak adom, türelméért és hozzáértő segítségéért, amely tanácsok nagyban segítettek, és a lehető legmagasabb szintre emelték dolgozatom elfogadhatóságát, minőségét.

Tisztelettel és köszönettel:

Dinnyés Balázs Emil